INTERNSHIP REPORT

PFE - Final Year Project

# SPRINT 3

## HTML, CSS, Tailwind & JavaScript

**Full Name :** Ayoub Barahal

**Duration :** 1 Week

**Status :** 100% Completed

*"Mastering front-end fundamentals for web development."*

# Table des matières

**1 Introduction** **3**

**2 HTML** **3**
2.1 HTML Presentation . . . 3
2.2 Document Structure . . . 3
2.3 Metadata . . . 3
2.4 Semantic HTML . . . 3
2.5 Headers and Sections . . . 3
2.6 Attributes . . . 3
2.7 Basic Text Principles . . . 3
2.8 Links . . . 3
2.9 Lists . . . 4
2.10 Navigation . . . 4
2.11 Tables . . . 4
2.12 Forms . . . 4
2.13 Images . . . 4
2.14 Audio and Video . . . 4
2.15 Model, Positioning and Shadow . . . 4
2.16 HTML API . . . 4
2.17 Debugging . . . 4
2.18 Other Inline Text Elements . . . 4
2.19 Details and Summary . . . 4
2.20 Dialog Box . . . 5

**3 CSS** **5**
3.1 Basics and CSS Organization . . . 5
3.2 Colors, Sizes and Spacing . . . 5
3.3 Layout . . . 5
3.4 Typography and Text . . . 5
3.5 Borders and Backgrounds . . . 5
3.6 Interactions and Animations . . . 6
3.7 Lists and Counters . . . 6
3.8 Overflow and Scrolling . . . 6
3.9 Functions and Advanced Effects . . . 6
3.10 User Interactions . . . 6
3.11 Variables and Customizations . . . 6

**4 Tailwind CSS** **6**
4.1 Layout & Box Model . . . 6
4.2 Flexbox & Grid . . . 7
4.3 Spacing (Margin & Padding) . . . 7
4.4 Sizing . . . 7
4.5 Typography . . . 7
4.6 Backgrounds & Borders . . . 8
4.7 Effects & Filters . . . 8
4.8 Transitions & Animations . . . 8
4.9 Transforms . . . 9

5.4 Control Flow . . . . . 10
5.5 Functions . . . . . 10
5.6 Objects . . . . . 10
5.7 Collections . . . . . 10
5.8 Classes . . . . . 10

**6 Conclusion** **10**

# 1 Introduction

This Sprint focused on mastering the core front-end technologies : HTML for structure, CSS & Tailwind for styling, and JavaScript for interactivity. The objective was to build responsive and accessible web pages following best practices.

# 2 HTML

## 2.1 HTML Presentation

HTML is the basic language used to create web pages. It serves to structure content and make it understandable by browsers.

## 2.2 Document Structure

An HTML document has a hierarchical structure : `<!DOCTYPE html>`, `<html>`, `<head>` and `<body>`. This structure allows for proper organization of content and metadata.

## 2.3 Metadata

`<meta>` tags provide information about the page, such as encoding, keywords or description, useful for browsers and search engines.

## 2.4 Semantic HTML

Semantic elements (`<header>`, `<footer>`, `<article>`, `<section>`, etc.) give meaning to content, facilitating understanding for users and assistive technologies.

## 2.5 Headers and Sections

Headings `<h1>` to `<h6>` structure content and create a logical hierarchy. Sections allow dividing content into coherent parts.

## 2.6 Attributes

Attributes add information to HTML elements. Some are global (`id`, `class`, `style`), others are specific (`src` for `<img>`, `href` for `<a>`).

## 2.7 Basic Text Principles

HTML allows text formatting : bold `<strong>`, italic `<em>`, quotes `<blockquote>`, etc.

## 2.8 Links

Links `<a>` allow navigation to other pages or sections, and their text must be clear for accessibility.

## 2.9 Lists

Lists organize content in a structured way : ordered lists `<ol>`, unordered lists `<ul>` and description lists `<dl>`.

## 2.10 Navigation

Navigation is essential to guide the user. `<nav>` elements and structured menus facilitate access to website sections.

## 2.11 Tables

Tables `<table>` serve to present tabular data in a clear and readable way.

## 2.12 Forms

Forms `<form>` allow users to input data. Fields must be properly labeled for accessibility.

## 2.13 Images

Images `<img>` enrich content. They must have alternative text `alt` to be accessible.

## 2.14 Audio and Video

Elements `<audio>` and `<video>` allow media integration, with user controls and text alternatives for accessibility.

## 2.15 Model, Positioning and Shadow

CSS combined with HTML allows managing appearance, layout and visual effects like shadows or positioning.

## 2.16 HTML API

JavaScript can interact with HTML via the DOM to make pages dynamic and interactive.

## 2.17 Debugging

Keyboard navigation relies on tab order and focus. Elements must be organized logically for a fluid experience.

## 2.18 Other Inline Text Elements

Other tags allow structuring and enriching text (`<span>`, `<mark>`, `<abbr>`, etc.).

## 2.19 Details and Summary

Elements `<details>` and `<summary>` create collapsible sections, useful for additional information without cluttering the page.

### 2.20 Dialog Box

The `<dialog>` element allows creating pop-ups or dialog boxes accessible directly via HTML.

## 3 CSS

### 3.1 Basics and CSS Organization

- **Box Model :** Every HTML element is a box composed of `content`, `padding`, `border`, `margin`. Understanding this structure is essential for managing sizes, spacing and layout.
- **Selectors :** Choosing which element to style (`tag`, `class`, `id`, `pseudo-classes`, `pseudo-elements`).
- **Nesting :** Allows writing organized and readable rules, especially with preprocessors.
- **Cascade and Specificity :** When multiple CSS rules apply, the browser decides which one to use based on priority, specificity and order.
- **Inheritance :** Some properties are passed to children if not redefined.

### 3.2 Colors, Sizes and Spacing

- **Color :** Define `color`, `background-color`, gradients, transparencies.
- **Size Units :** px, em, rem, %, vw, vh, etc.
- **Spacing :** `margin` and `padding` to manage space between boxes and content.

### 3.3 Layout

- **General Layout :** Methods to organize elements : normal positioning, flexbox, grid, logical properties.
- **Flexbox :** Layout on a single dimension (row or column), useful for aligning and distributing space between elements.
- **Grid :** Two-dimensional layout, organizes in rows and columns.
- **Logical Properties :** Automatic adaptation according to text direction (`inline`, `block`).

### 3.4 Typography and Text

- **Text and Fonts :** `font-family`, `font-size`, `font-weight`, `line-height`.
- **Pseudo-elements and Pseudo-classes :** `::before`, `::after`, `:hover`, `:focus` to style without modifying HTML.
- **Shadows :** `box-shadow` and `text-shadow` to give relief to elements.

### 3.5 Borders and Backgrounds

- **Borders :** Style (`solid`, `dashed`), size, color, radius (`border-radius`).
- **Gradients :** Linear and radial gradients for element backgrounds.
- **Backgrounds :** `background-color`, `background-image`, `background-position`, `background-size`.

### 3.6 Interactions and Animations

- **Transitions :** Smooth animation between two states of an element (`transition-property`, `duration`, `timing-function`, `delay`).
- **Animations :** Create continuous effects with `@keyframes`.
- **SPA Transitions :** Visually transition from one page to another for single-page applications, with `document.startViewTransition()`.

### 3.7 Lists and Counters

- **HTML Lists :** `<ul>`, `<ol>`, `<dl>`
- **List Styles :** Numbering, letters, custom bullets, `list-style-type`, `::marker`.
- **CSS Counters :** `counter-reset`, `counter-increment`, `counter()` to automatically number elements.

### 3.8 Overflow and Scrolling

- **Overflow :** Manage content that overflows (`visible`, `hidden`, `scroll`, `auto`, `clip`).
- **Scrolling :** Control with `overflow-x`, `overflow-y`, `scroll-behavior`, `overscroll-behavior`.
- **Scroll Snap :** Align content to specific points when scrolling.

### 3.9 Functions and Advanced Effects

- **Filters :** Graphic effects (`blur`, `brightness`, `contrast`).
- **Blend Modes :** Element blending (`mix-blend-mode`, `background-blend-mode`).
- **Paths, Shapes, Clipping, Masking :** Create complex shapes and manage element visibility.
- **CSS Functions :** `calc()`, `min()`, `max()`, `clamp()` to dynamically calculate values.

### 3.10 User Interactions

- **Cursors and Pointers :** `cursor` to indicate interactivity.
- **Z-index and Stacking :** Control which element is above or below.
- **Focus and Accessibility :** Keyboard focus management, pseudo-classes `:focus`, `:focus-within`.
- **Popovers and Dialog Boxes :** Interactive elements for tooltips, alerts or messages.

### 3.11 Variables and Customizations

- **Custom Properties (Variables) :** Reuse values with `-variable-name` and `var()`.
- **Container Queries :** Adjust an element according to its container size, not just the window.

## 4 Tailwind CSS

### 4.1 Layout & Box Model

| Property | Simple Explanation | Tailwind Example |
|---|---|---|
| aspect-ratio | Defines width/height ratio of an element | aspect-w-16 aspect-h-9 |
| columns | Creates multiple columns for text | columns-3 |
| break-after | Where an element should cause a page break | break-after-auto, break-after-page |
| break-before | Break before element | break-before-auto |
| break-inside | Prevents break inside an element | break-inside-avoid |
| box-decoration-break | How borders/margins behave on multiple lines | box-decoration-slice |
| box-sizing | How width/height is calculated | box-border, box-content |
| display | Defines display type | block, inline, flex, grid |
| float | Make element float | float-left, float-right, float-none |
| clear | Prevents overlap of floats | clear-left, clear-right, clear-both |
| isolation | Creates blend context | isolate, isolation-auto |
| object-fit | How image/video fills its container | object-cover, object-contain |
| object-position | Positions image in its container | object-center, object-top |
| overflow | How to manage overflow | overflow-auto, overflow-hidden |
| overscroll-behavior | Scroll behavior when limit reached | overscroll-auto, overscroll-contain |
| position | Type of position | static, relative, absolute, fixed, sticky |
| top/right/bottom/left | Position relative to parent | top-0, left-4 |
| visibility | Visible or hidden | visible, invisible |
| z-index | Stacking order | z-10, z-50 |

## 4.2 Flexbox & Grid

## 4.3 Spacing (Margin & Padding)

## 4.4 Sizing

## 4.5 Typography

- font-family : font-sans, font-serif
- font-size : text-sm, text-2xl
- font-weight : font-bold, font-light
- line-height : leading-tight, leading-loose
- text-align : text-left, text-center
- text-color : text-red-500, text-gray-700

| Property | Simple Explanation | Tailwind Example |
|---|---|---|
| flex-basis | Initial width of a flex | basis-1/2 |
| flex-direction | Direction of children | flex-row, flex-col |
| flex-wrap | Allows wrapping to new line | flex-wrap, flex-nowrap |
| flex | Flexible size | flex-1, flex-auto |
| flex-grow | Relative growth | grow, grow-0 |
| flex-shrink | Relative shrinking | shrink, shrink-0 |
| order | Order of elements | order-1, order-last |
| grid-template-columns/rows | Define columns/rows | grid-cols-3, grid-rows-2 |
| grid-column/row | Element position | col-span-2, row-start-1 |
| grid-auto-flow | Automatic filling | grid-flow-row, grid-flow-col |
| gap | Spacing between cells | gap-4 |
| justify-content/items/self | Horizontal alignment | justify-start, items-center, self-end |
| align-content/items/self | Vertical alignment | content-center, items-start, self-center |
| place-content/items/self | Combines align and justify | place-content-center, place-items-start |

| Property | Tailwind Example |
|---|---|
| padding | p-4, px-2, py-3 |
| margin | m-4, mx-2, my-3 |

- text-decoration : underline, line-through
- text-transform : uppercase, capitalize
- text-overflow : truncate

## 4.6 Backgrounds & Borders

- background-color : bg-blue-500
- background-image : bg-gradient-to-r from-red-500 to-yellow-500
- border-width : border, border-2
- border-color : border-red-500
- border-radius : rounded, rounded-lg
- outline : outline-none, outline-white

## 4.7 Effects & Filters

- box-shadow : shadow, shadow-lg
- opacity : opacity-50
- mix-blend-mode : mix-blend-multiply
- filter & backdrop-filter : blur, brightness-125, contrast-150

## 4.8 Transitions & Animations

- transition-property : transition-all, transition-colors

| Property | Tailwind Example |
|---|---|
| width | w-1/2, w-full |
| min-width | min-w-0, min-w-full |
| max-width | max-w-xs, max-w-screen-lg |
| height | h-16, h-full |
| min-height | min-h-0, min-h-screen |
| max-height | max-h-96, max-h-screen |

— transition-duration : duration-300
— transition-timing-function : ease-in, ease-out
— animation : animate-bounce, animate-spin

### 4.9 Transforms

— rotate : rotate-45
— scale : scale-110
— translate : translate-x-4, translate-y-2
— skew : skew-x-12

### 4.10 Interactivity & Accessibility

— cursor : cursor-pointer, cursor-not-allowed
— pointer-events : pointer-events-none
— user-select : select-none
— accent-color : accent-red-500
— color-scheme : dark

## 5 JavaScript

### 5.1 JavaScript Presentation

— Client-side programming language (and server-side with Node.js).
— Allows making web pages interactive and dynamic.
— Integrated in browsers and easy to execute with `<script>`.

### 5.2 Types and Data Structures

— **Primitive Types :** Number, String, Boolean, Null, Undefined, Symbol, BigInt.
— **Complex Structures :** Objects, Arrays (Array), Map, Set.
— Allow storing and manipulating data in different ways.

### 5.3 Comparison Operators

— Value comparison : `==`, `===`, `!=`, `!==`.
— Numerical or lexical comparison : `<`, `>`, `<=`, `>=`.
— Strict comparisons (`===`) check both value and type.

### 5.4 Control Flow

- **Conditions :** `if`, `else if`, `else`, `switch`.
- **Loops :** `for`, `while`, `do...while`, `for...of`.
- Allow executing code based on conditions or repeating actions.

### 5.5 Functions

- Reusable code blocks to perform tasks.
- **Classic Declaration :** `function myFunc() {}`
- **Arrow Function (ES6) :** `const myFunc = () => {}`
- Possibility to pass arguments and return values.

### 5.6 Objects

- Collections of **key-value pairs**.
- Allow representing real entities with properties and methods.
- Access values via `obj.key` or `obj["key"]`.
- Possibility to add, modify or delete properties dynamically.

### 5.7 Collections

- **Indexed :** Arrays (Array), TypedArray.
- **Keyed :** Map, WeakMap, Set, WeakSet.
- Allow managing data sets, with or without order and with different constraints (uniqueness, referenceability, etc.).
- ES6 Operators : **Spread (...)** and **Rest (...)** to easily manipulate collections.

### 5.8 Classes

- Templates for creating objects with **properties and methods**.
- **Inheritance :** `extends` allows a child class to inherit methods and properties from a parent class.
- **Private fields (#)** for secure properties.
- **Static fields/methods** for members shared by all instances.
- **Static initialization blocks** to dynamically initialize static fields.
- Classes allow clearer structure and reusable code.

## 6 Conclusion

Sprint 3 enabled :

- Understanding and structuring semantic HTML,
- Effective styling with CSS and Tailwind,
- Creating dynamic pages with JavaScript.

The acquired skills prepare for creating interactive and responsive websites.